

EduVantage: An Intelligent Graduate Recommender System

Project Team

Muhammad Arham Akmal	21I-0710
Mudasir Saeed	21I-0592
Umair Tanveer	21I-0431

Session 2021-2025

Supervised by

Mr. Majid Hussain

Co-Supervised by

Dr. Zeshan Khan



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	UI/UX Development Module	3
1.3.2	User Information Portal	3
1.3.3	CV Upload for Profile Auto-Population	4
1.3.4	Academic Data Scraper	5
1.3.5	NLP-Based Query Analysis	6
1.3.6	Smart Data Identifying by utilizing NER	6
1.3.7	LLM Precision Tuning of Processed Data	7
1.3.8	Priority URL Retrieval Engine	8
1.3.9	Application Deadline Tracker	8
1.3.10	Database Management and Data Storage (MongoDB)	9
1.4	User Classes and Characteristics	10
2	Project Requirements	11
2.1	Use-case/Event Response Table/Storyboarding	11
2.1.1	Detailed Use Cases	12
2.2	Functional Requirements	19
2.2.1	UI/UX Development	19
2.2.2	User Information Portal	19
2.2.3	Academic Data Scraper	19
2.2.4	Smart Data Identifier	20
2.2.5	NLP-based Query Analysis	20
2.2.6	LLM Precision Tuning of Processed Data	20
2.2.7	Priority URL Retrieval Engine	20
2.2.8	Application Deadline Tracker	21
2.2.9	Database Management and Data Storage	21
2.3	Non-Functional Requirements	21
2.3.1	Reliability	21
2.3.2	Usability	22

2.3.3	Performance	22
2.3.4	Security	22
2.4	Domain Model	22
3	System Overview	25
3.1	Architectural Design	25
3.1.0.1	Architectural Layout	25
3.2	Design Models	26
3.2.1	Activity Diagrams	26
3.2.2	System-level Sequence Diagram	32
3.2.3	State Transition Diagram	39
3.2.4	Data Flow Diagram	40
3.3	Data Design	42
	References	45

List of Figures

2.1	Use Case Diagram	11
2.2	Domain Model Diagram of EduVantage	23
3.1	Architecture Design	26
3.2	Activity Diagram for Search and view Recommendations	26
3.3	Activity Diagram for Upload CV and fill profile	27
3.4	Activity Diagram for Check Program Availability	28
3.5	Track application deadline	29
3.6	Compare Universities	30
3.7	Storing and Scraping Data	31
3.8	SSD of "Search and View Program Recommendations"0	32
3.9	SSD of "Upload CV and Autofill Profile"	33
3.10	SSD of "Check Program Eligibility"	34
3.11	SSD of "Track Application Deadlines"	35
3.12	SSD of "Compare Universities"	36
3.13	SSD of "Scrape and Store University Data"	37
3.14	State Transition Diagram	39
3.15	Data Flow Diagram	41
3.16	Entity Relationship Diagram	42

List of Tables

1.1	Identification of the user classes	10
2.1	Detailed use case for Search and View Program Recommendations	12
2.2	Detailed use case for Upload CV and Auto fill Profile	13
2.3	Detailed use case for Scrape and Store University Data	14
2.4	Detailed use case for Check Program Eligibility	15
2.5	Detailed use case for Program Fit Score	16
2.6	Detailed use case for Track and Notify Application Deadlines	17
2.7	Detailed use case for View and Compare Programs	18

Chapter 1

Introduction

This project proposal will describe the intended objectives, features and technical architecture of **EduVantage** a full-fledged web-based platform to guide graduates on selecting appropriate universities for their higher studies. It is also important because in today's global world there are so many factors to consider when selecting a university and its masters program like country preferences, admission criteria, test requirements for the specific course relevant qualifications. The conventional way of manually checking colleges and universities is time-consuming, can leave you with incomplete information and leaves a lot to be desired when it comes to the final deployment of your choices.

Our project provides a cutting-edge solution to these issues by using artificial intelligence to automate and customize university search. Our platform takes real-time data from a range of university websites and employs modern technologies like Natural Language Processing (NLP) and Named Entity Recognition (NER) to make personalized suggestions. In addition, we apply a Large Language Model (LLM) to improve personalization by explaining why specific universities are a suitable fit based on user profiles.

1.1 Problem Statement

In the digital age, graduate students face an increasingly challenging process in picking the correct university for their master's degrees. With thousands of universities worldwide, each providing a different program with its own admission criteria, deadlines, and eligibility requirements, the process has become more complex and time-consuming than ever before. The traditional method of manually looking for institutions, reviewing program details, and comparing possibilities is not only inefficient, but it often leads to errors and missed chances. This information overload can overwhelm students, resulting in rash or inadequately researched judgments and, in some circumstances, missing critical application deadlines entirely.

Furthermore, many students struggle to grasp how their distinct academic profiles fit into various university programs. They lack direction on how their GPA, test scores, and academic history align with the requirements of various colleges. The process is further hampered by the lack of personalization in existing platforms, which do not offer specialized recommendations or reasons for why various universities may be a suitable fit.

There is a strong demand for an automated, intelligent solution that may simplify the university selection process by tailoring recommendations to each student's academic profile and preferences. Such a system should not only screen appropriate programs, but also provide real-time updates on changing deadlines and admission requirements, giving students the information they need to make sound judgments.

1.2 Scope

This initiative seeks to transform the higher education search process by focusing on the top 150 universities in the United States, United Kingdom, and the European Union[12]. It would provide users with highly personalized recommendations based on their specific profiles, such as academic achievements, exam results, and job goals. The system will evaluate and recommend the most appropriate university programs based on each user's individual requirements. It will also make thorough scholarship recommendations to help customers identify appropriate financial aid opportunities. To guarantee that users have access to the most up-to-date information, the platform will continuously monitor university websites for updates and instantly refresh its database to reflect any changes in program specifics, admission requirements, or deadlines. Users will benefit from complete comparisons of universities, which include program options, fees, and rankings. An interactive preference query system will enable users to dynamically refine their criteria, allowing for more precise and informed decisions about their educational routes.

1.3 Modules

- **UI/UX Development:** Creating the basic UI design for our web-app, ensuring a smooth and interactive user experience.
- **User Information Portal:** This module serves as the entry point to the whole system as users can input, update and manage their personal details as well as their academic details.
- **CV Upload for Profile Auto-Population:** Enables the users to submit their CV or resume which will be used to automatically populate their profile.

- **Academic Data Scraper:** This module will scrape unstructured data from university websites which will be further processed
- **Smart Data Identifying by utilizing NER:** Unstructured data after the initial scraping process will be structured using Named Entity Recognition (NER).
- **NLP-Based Query Analysis:** User queries will be processed using Natural Language Processing (NLP) techniques to simplify and better understand what the user is demanding from the system.
- **LLM Precision Tuning of Processed Data:** This module uses a fine-tuned LLM to calculate the eligibility of a specific user with a specific university by calculating a match score between the user-provided information and the university requirements.
- **Priority URL Retrieval Engine:** Searches for relevant URLs using the user queries, ranks the resulting URLs to increase efficiency in the data scraped.
- **Application Deadline Tracker:** Monitors university websites to look for any changes in the application deadlines, also alerts users using node-mailer [8]
- **Database Management and Data Storage (MongoDB):** This module outlines the design and implementation of the project's database.

1.3.1 UI/UX Development Module

Focuses on creating a responsive and user-friendly interface for our web-app platform using the MERN stack and utilizing other front-end frameworks such as Material-UI [3] and Bootstrap [5]

1. A user-centered design to enhance accessibility and ease navigation for the user.
2. Responsiveness to ensure seamless transition when using the web-app on different devices and on different screen sizes.
3. Interactive UI for the user to input their data and for their query submission.
4. Frontend development done using React, with Express and Node.js on the backend.

1.3.2 User Information Portal

1. User Registration and Authentication

- (a) Users can register their account via their email or even other social media accounts (i.e Facebook, Google).
- (b) Users will be authenticated using JWT [7] (Json Web Token) to ensure that users data is protected.

2. Profile Creation and Management

- (a) Users can manage their profiles and input personal details (name, contact no) and their academic details (GPA, previous institutions, degrees) and their previous test scores
- (b) A progress meter will be implemented to show the users their progress in completely setting up their profile.

3. Eligibility Status Display

- (a) The system will calculate and display a specific user's eligibility status based on their academic credentials (i.e GPA, Degrees, Test Scores)
- (b) The eligibility status will be dynamically changed whenever a user changes their information

1.3.3 CV Upload for Profile Auto-Population

This module allows the user to upload their CVs in PDF format to then extract the information in that PDF document and use that information to populate a user's profile.

1. CV Upload Functionality

- (a) The upload functionality will be implemented by using React.js with File-Pond. [9]
- (b) CV files can be temporarily be stored in our MongoDB database using multer. [2]

2. File Parsing and Data Extraction from the PDFs

- (a) PyPDF2 or pdfplumber will be used for extracting the data out of the PDFs.
- (b) The data will then be cleaned and unnecessary data (i.e footers, headers, special characters) will be removed.

3. NER for Information Classification

- (a) spaCy and a custom NER Model can be used to extract specific text from the PDF files. spaCy is a powerful Python library for NLP with pre-trained models to handle this task.

4. Mapping the Extracted data to the User Profile

- (a) The user profile information will be stored in our MongoDB database.
- (b) We will be using JOI.js [10] to ensure that the extracted data is mapped to specific fields where the data should be ideally going.

1.3.4 Academic Data Scraper

The Academic Data Scraper uses puppeteer [11] to gather information from university websites.

1. URL Identification and Web Crawling

- (a) Puppeteer [11] which is a headless browser tool specifically for data scraping as well as data crawling will be used to navigate university websites and to find relevant sub-URLs.
- (b) Puppeteer will be used to simulate user interactions, such as clicking on specific fields such as 'Deadlines' or 'Programs'.

2. Scraping Relevant Academic Data

- (a) Relevant pages will be identified through the web crawling process and details such as program name, fees, deadlines will be scraped from those pages.
- (b) This data will be stored in a structured format such as JSON.

3. Handling Pagination and Dynamic Content

- (a) Puppeteer will once again be used to handle the dynamic loading of data such as pagination or scrolling. Puppeteer will stimulate user actions to load this hidden data.
- (b) In pages where the content is loaded via AJAX, we will wait for the page to load completely.
- (c) Pagination will be detected and afterwards handled (i.e by clicking 'Next' to load more data).

4. Structuring and Storing the Scraped Data

- (a) JSON format will be used to structure the data before saving it to the database.
- (b) The scraped data will be converted into a structured format (JSON) with fields for fees, deadlines, programs etc.

1.3.5 NLP-Based Query Analysis

Using techniques like Tokenization, Parts-of-Speech Tagging(POS), Named Entity Recognition (NER) and using LLM, this NLP-Based module will be implemented.[13]

1. User Query Processing and Keyword Extraction
 - (a) Using NLP techniques such as NER the system will extract relevant entities and keywords from the user prompt/input. This will allow user to input his prompt without the need of a specific template.
2. Profile-Based Keyword Extraction
 - (a) Using Tokenization and POS tagging, the system will also extract relevant information from the user profile such as CGPA, Bachelors, Preference of region and universities etc if such entities and useful information is not written by the user in the prompt they are giving.
 - (b) If the user overlaps this information within the query and makes changes to the entities the system will make sure to cross-check and ensure the most updated data (The user query) is considered.
3. Automated Prompt Generation Now we have the extracted keywords from the user query and profile these unstructured keywords will be given to the LLM for prompt generation. The pre-trained LLM will take this set of keywords and generate a prompt again which will be given as input to the fine-tuned LLM for searching/Analysis for providing user universities that best fit the users profile and preferences.

1.3.6 Smart Data Identifying by utilizing NER

The goal of this module is to separate organized data from unstructured data that has been scraped from university websites. A modified version of BERT will be utilized by the NER model to extract pertinent items from unstructured text, including degree programs, costs, deadlines, and entrance requirements.

1. BERT Fine-Tuning for NER
 - (a) Fine-tune a BERT-based model using a manually annotated dataset.
 - (b) Annotate data to identify relevant terms like "Program Name," "Fee," "Application Deadline," and "Admission Requirements."
2. Handling Unstructured Data

- (a) The NER model will process unstructured data scraped from universities websites and tag pertinent data points in the unstructured text and transform them to structured fields (e.g., degree name, cost, location, deadline).

3. Continuous Model Improvement

- (a) Periodic re-training and fine-tuning with new annotated data when university websites evolve or new programs are added.

1.3.7 LLM Precision Tuning of Processed Data

It focuses on fine-tuning the Large Language Model (LLM), specifically improving its accuracy in determining the optimum fit for users based on their academic profile, preferences, and university requirements. The purpose of this module is to ensure that the LLM interprets user queries correctly and ranks colleges based on the tailored fit score, utilizing the structured data generated by the NER model.

1. Initial Fine-Tuning on Processed Data

- (a) Fine-tuning LLaMA or GPT-based models: Fine-tune a pre-trained LLM (such as LLaMA, GPT-3, etc.) with structured data (e.g., program information, requirements) and user profiles (GPA, test scores, preferences).

2. Fit Score Calculation

- (a) The LLM is trained to generate the fit score by weighting significant variables (such as GPA, exam results, and experience) [4].
- (b) During training, the LLM learns how to normalize user input (such as GPA or test scores) so that it may be properly compared to program criteria.
- (c) The model is designed to handle various combinations of user criteria, dynamically modifying the fit score in response to new input.

3. Ranking and Sorting of Programs

- (a) The fine-tuned LLM ranks university programs in real time using the fit score. The rating will change dynamically when the user adds new preferences or updates their profile.
- (b) The model gives a natural language explanation for each fit score, indicating why some programs are a good fit. Like the following: "Your GRE score of 320 is above the minimum requirement of 310 for University X."

4. Integration with Retrieval-Augmented Generation (RAG)

- (a) RAG provides additional instructions to the LLM to ensure that responses match the most recent database information and user-specific demands [1].
- (b) Incorporate external knowledge sources, including updated program specifics, new university admissions criteria, and changing dates, into the LLM's response.

1.3.8 Priority URL Retrieval Engine

This module focuses on retrieving relevant URLs and data based on the user queries and profile information, specifically when the user selects the 'deep analysis' option

1. Query Simplification Using LLM

- (a) After the user writes a prompt and selects 'deep analysis' this prompt as well as the user data are sent to an LLM to simplify this query and generate a more refined query to use on the search engine.
- (b) GPT-4 will be used to generate a new refined query from the user query already provided.

2. TF-IDF Scoring for URL Ranking

- (a) URLs will be ranked on the basis of their Term Frequency-Inverse Document Frequency (TF-IDF) scoring.
- (b) URLs will be ranked if they have a TF-IDF higher than 0.5.
- (c) Scikit-learn [6] will be used to rank the web pages by implementing the TF-IDF vectorization.

3. Web Crawling to Identify Sub-URLs

- (a) Puppeteer will be used for web crawling to discover sub-URLs which are more in tune with the user query to scrape specific data.
- (b) Specific sub-URLs will be added to a list for further extraction.

4. Data will be scraped from these sub-URLs NER will be applied on this data and the resulting data will be used for further data processing and LLM Fine-Tuning.

1.3.9 Application Deadline Tracker

This module is responsible for tracking application dates on university websites and reminds users when application dates are closing via email.

1. Tracking Last-Modified HTTP Header

- (a) Puppeteer will be used to fetch the Last-Modified header from university sites to check if the page has been updated.
- (b) A stored timestamp will be compared with the newly retrieved timestamp to check if the webpage has been updated.

2. Rescraping Updated Web Pages

- (a) Puppeteer will be used to resrape the pages where the Last-modified date has been changed.
- (b) The updated deadline is then stored in our database.

3. Notification for deadline alerts

- (a) When a new deadline is detected and is used to replace the existing one, The system sends the user timely reminders using nodemailer.

4. All the extracted and updated deadlines are stored in a NoSQL MongoDB database.

1.3.10 Database Management and Data Storage (MongoDB)

This module outlines the design and implementation of the project's database.

- 1. MongoDB's document oriented structure is used to store user profiles, university details, programs, fees, deadlines etc. in the form of collections/models.
- 2. There will be different collections for Users, Universities, Programs and Deadlines.
- 3. Relationships will be established between different collections using references.

1.4 User Classes and Characteristics

User class	Description
Graduate Students	This typically caters for graduate students, that is, students seeking admission to master's programs. Master's programs are usually found in overseas countries. The student may come from any branch of engineering or science or humanities. The graduates will have exposure to online applications and systems, though their technical level could be very different. They generally look at programs that align with their academic profiles-GPA and test scores- and personal preferences-location and fees. The system, they use to get tailor-made suggestions, track deadlines for applications, and compare different courses in making an informed decision.
Academic Advisor	An academic advisor is a highly knowledgeable professional who guides a student through his or her graduate application journey. The advisors help students bring their academic and career goals in line with the most appropriate programs through fit score interpretation and program comparisons. Advisors use the system to access student profiles, track application deadlines, and recommend programs based on eligibility and preferences. They play an important role in counseling students on appropriate academic decisions and help students meet deadlines and program requirements.
University Admissions Officers	University admissions officers represent various academic institutions and are responsible for managing and reviewing student applications. Their interest lies in ensuring that the programs offered by their institution are accurately represented in the EduVantage system. They want to attract qualified and eligible students who meet the program's criteria, which helps maintain high acceptance rates. Officers may also use the platform to track trends in student preferences and ensure that program details such as fees, deadlines, and admission requirements are kept up to date.
Parents/ Guardians of Students	Parents or guardians of students play a supportive role in their child's higher education journey. They may not have the same level of technical expertise as their children but are still invested in researching programs and keeping track of application deadlines. Parents are particularly concerned about financial factors such as tuition fees, scholarships, and overall program costs. They use the system to assist their children in comparing programs and ensuring that application deadlines are met.

Table 1.1: Identification of the user classes

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

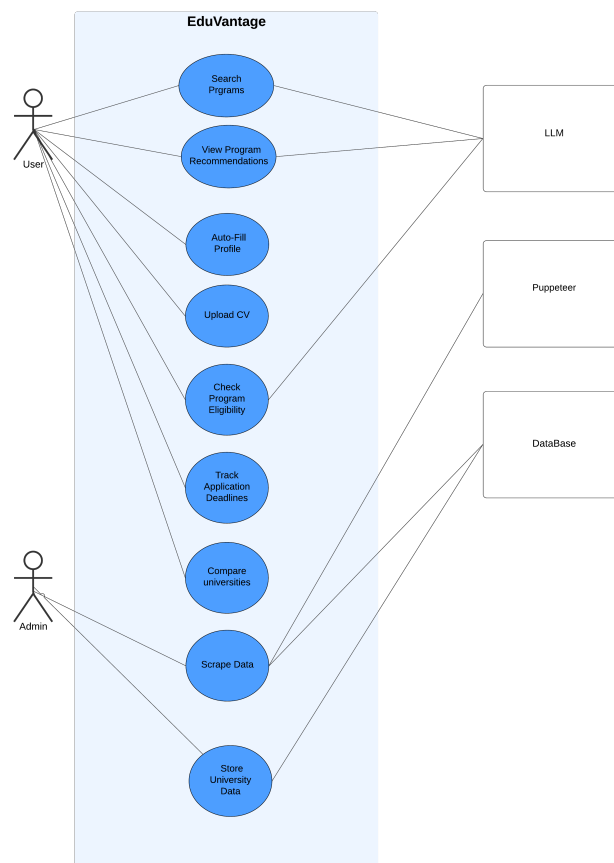


Figure 2.1: Use Case Diagram

2.1.1 Detailed Use Cases

Use Case Name	Search and View Program Recommendations
Scope	EduVantage System
Level	User Goal Level
Primary Actor	Student (user)
Stakeholders and Interest	Student: Wants accurate and up to date information for graduate/master programs that fit their academic profile and preferences. Academic Advisor: Wants to ensure that students receives accurate and up to date, tailored program recommendation that fits the student's academic profile and preferences. Universities: Wants their programs to be accurate and up to date.
Preconditions	The Student has logged in their account. The Student has completed their profile, filling in the required information.
Success Guarantee/ Postcondition	The Student is provided with a list of graduate programs, which are ranked by fit score, and matches the Student profile and preferences.
Main Success Scenario	1. The Students logs in their account 2. The Student has their profile already made or fills in the required information to complete their profile. 3. The system processes the input using a LLM model 4. The system will retrieve relevant program information from the database 5. The System will calculate the fit score for each of the relevant programs 6. The System will display a ranked list of programs based on the fit score. 7. The Student will view the programs suggested.
Extensions	1. The system detects missing, incomplete or invalid student data in student profile 2. The system finds no programs that meet the student's criteria.
Special Requirements	1. The System returns the results within 1 minute to the user, to ensure a smooth experience. 2. The results must be relevant to the user, it should be accurate and up to date.
Technology and Data Variation	The user may input their profile details via web forms or upload a CV for automatic data extraction.

Table 2.1: Detailed use case for Search and View Program Recommendations

Use Case Name	Upload CV and Auto-Fill Profile
Scope	EduVantage System
Level	Sub-function level
Primary Actor	Student (user)
Stakeholders and Interest	Student: Wants an easy way to populate their profile without the need for manual input. Admin: Wants to insure that the data extraction process is accurate and consistent with system requirements.
Preconditions	The Student has logged in their account. The CV is in a supported format (PDF).
Success Guarantee/ Postcondition	After data extraction from the students CV, it is automatically filled in the relevant fields in the Student profile.
Main Success Scenario	1. The Students logs in their account 2. The Student goes to their profile section and uploads their CV in the correct format (PDF). 3. The system processes the uploaded CV. 4. The system will extract relevant information from the CV and auto fills the Student profile with the relevant information. 5. The Student reviews and confirms the auto-filled fields.
Extensions	1. The students uploads an unsupported file format. 2. The system is unable to extract certain details from the CV.
Special Requirements	1. The System must support file uploads of at max 5MB in size. 2. The results must be relevant to the user, it should be accurate and up to date.
Technology and Data Variation	The user may input their profile details via web forms or upload a CV for automatic data extraction.

Table 2.2: Detailed use case for Upload CV and Auto fill Profile

Use Case Name	Scrape and Store University Data
Scope	EduVantage System
Level	System function level
Primary Actor	Admin/System
Stakeholders and Interest	Universities: Wants their program information to be correct and up to date. Students: Wants to rely on accurate and updated information. Admin: Wants to insure that the scraped data is accurate and up to date.
Preconditions	1. The scraping process is triggered manually or scheduled to run automatically. 2. University websites must be accessible and allow scraping
Success Guarantee/ Postcondition	The system successfully scrapes and stores up-to-date program data
Main Success Scenario	1. The admin/system triggers the scraping process using Puppeteer. 2. The system goes to the relevant sites and scrapes relevant data such as program names, fees, and deadlines. 3. The scraped data is processed by the NER model to structure the information. 4. The structured data is stored in the MongoDB database. 5. The admin will receives a report of logs confirming successful scraping and storage.
Extensions	1. The students uploads an unsupported file format. 2. The system is unable to extract certain details from the CV.
Special Requirements	1. The website blocks scraping attempts
Technology and Data Variation	1. Scraping may be triggered manually by the admin or scheduled to run at regular intervals. 2. Data format from university websites may vary, requiring adaptive scraping algorithms

Table 2.3: Detailed use case for Scrape and Store University Data

Use Case Name	Check Program Eligibility
Scope	EduVantage System
Level	User Goal Level
Primary Actor	Student (User)
Stakeholders and Interest	Students: Wants to verify if they meet the criteria for a specific university. Academic Advisors: Wants to insure that student apply to programs they are eligible for .
Preconditions	The Student is logged in and their profile is made with the information filled out.
Success Guarantee/ Postcondition	The student receives a list of programs for which they meet the eligibility requirements based on their academic profile.
Main Success Scenario	1. The student logs into the EduVantage platform. 2. The student has made their profile. 3. The student writes a prompt for a list of universities that they are eligible for. 4. The system returns a list of programs where the student meets the eligibility criteria. The student views the list of eligible programs.
Extensions	1. The student's profile is missing key information (e.g., GPA or test scores). 2. The student does not meet the criteria for any selected programs.
Special Requirements	1. The system should return eligibility results within 30 seconds for a smooth user experience.
Technology and Data Variation	Program data is retrieved from MongoDB and structured to match the eligibility requirements.

Table 2.4: Detailed use case for Check Program Eligibility

Use Case Name	Provide Program Fit Score
Scope	EduVantage System
Level	User goal level
Primary Actor	Student (User)
Stakeholders and Interest	Students: Wants to see a fit score that quantifies how well they match specific programs. Academic Advisors: Interested in helping students choose programs based on personalized fit scores.
Preconditions	1. The Student is logged in and profile is filled with the necessary information. 2. The program data has been processed and stored in the system's database.
Success Guarantee/ Postcondition	The student is provided with a fit score for each program, helping them understand how well they match each one.
Main Success Scenario	1. The student logs into the EduVantage platform and has his profile filled. 2. The student then gives the prompt to search for the programs they are best for. 3. The system calculates a fit score for each program using the student's profile data and program requirements. 4. The system ranks the programs by fit score. 5. The student views the ranked programs along with their fit scores.
Extensions	1. The student's profile is missing key information (e.g., GPA or test scores). 2. Multiple programs have the same fit score.
Special Requirements	1. The fit score calculation must consider multiple factors, including GPA, test scores, work experience, and location preferences. 2. The system should provide explanations for how each fit score is calculated (e.g., "Your GPA of 3.8 exceeds the program's minimum requirement of 3.5").
Technology and Data Variation	Fit score calculations are performed by the LLM model based on program data stored in MongoDB.

Table 2.5: Detailed use case for Program Fit Score

Use Case Name	Track and Notify Application Deadlines
Scope	EduVantage System
Level	Subfunction level
Primary Actor	Student (User)
Stakeholders and Interest	Students: Wants to receive reminders about important application deadlines. Admin: Wants to Ensures that the system provides accurate and timely reminders
Preconditions	The Student is logged in and allowed to be reminded for his selected programs.
Success Guarantee/ Postcondition	The student is notified of upcoming deadlines for the programs they are interested in
Main Success Scenario	1. The student logs into the EduVantage platform and selects the programs for which they need to be reminded of deadlines. 2. The system tracks the application deadlines for each of the selected program. 3. The student receives a reminder via email when the deadline is near. 4. The student is able to act on the reminder and submit applications on time.
Extensions	1. The deadline information in the system is outdated or unavailable. 2. The student disables email notifications
Special Requirements	1. The system should support email notifications. 2. Notifications should be sent 30 days, 7 days, and 1 day before the application deadline.
Technology and Data Variation	1. Deadline data is scraped from university websites and stored in MongoDB. 2. Notifications can be sent via email.

Table 2.6: Detailed use case for Track and Notify Application Deadlines

Use Case Name	View and Compare Programs
Scope	EduVantage System
Level	User goal level
Primary Actor	Student (User)
Stakeholders and Interest	Students: Wants to compare multiple programs based on fit score, ranking, and other key factors (e.g., fees, deadlines). Academic Advisors: Interested in helping students make informed comparisons between programs.
Preconditions	1. The student has already received a list of recommended programs with fit scores. 2. The program data (e.g., fees, deadlines) is up-to-date in the system.
Success Guarantee/ Postcondition	The student is able to compare multiple programs based on fit score and other relevant factors, making an informed decision on which programs to apply to.
Main Success Scenario	1. The student views a list of programs with fit scores. 2. The student selects multiple programs for comparison. 3. The system displays the selected programs side-by-side in a comparison table. 4. The student views the comparison and saves preferred programs for future reference.
Extensions	1. The student tries to compare programs without selecting any. 2. One or more selected programs have incomplete data (e.g., missing fees or deadlines).
Special Requirements	1. The comparison table should display important attributes such as fit score, tuition fees, location, application deadlines, and duration of the program. 2. The system should allow the student to export the comparison as a PDF or save it in their profile.
Technology and Data Variation	Comparison data is retrieved from MongoDB and presented in a table format within the EduVantage web app.

Table 2.7: Detailed use case for View and Compare Programs

2.2 Functional Requirements

2.2.1 UI/UX Development

1. **Design Interface:** Develop a dynamic and responsive user interface employing React.js that is successful across both online and mobile platforms.
2. **Smooth Navigation:** Assure smooth transitions between forms and pages, and preserve uniformity across all web pages.
3. **Accessibility Compliance:** Implement ARIA-compliant features to enhance accessibility for all users, ensuring that the interface is usable by people with various disabilities.

2.2.2 User Information Portal

1. **Profile Creation via CV Upload:** Permit users to establish and complete their profiles by allowing them to submit their resumes in PDF format, with pertinent fields automatically extracted and filled in.
2. **Manual Detail Input:** Permit users to manually provide extra information, such as employment history, education, and certifications.
3. **Profile Update and Deletion:** Give users the option to edit or remove information from their profiles whenever they choose.

2.2.3 Academic Data Scraper

1. **Data Scraping from Universities:** Gather crucial academic information by scraping 150 specific university domains, paying particular attention to program specifics, due dates, and qualifying standards.
2. **Pagination Management:** Effectively manage pagination to retrieve data from parts that take up several pages.
3. **Data Recollection Scheduling:** Update data that has been scraped by periodically rescraping university websites (weekly or monthly, for example) to make sure the information is up to date.

2.2.4 Smart Data Identifier

1. **Entity Recognition and Extraction:** Use a pre-trained NER model to identify and extract key information such as program details, costs, and deadlines from unstructured data.
2. **Data Validation and Formatting:** Validate extracted data for accuracy and logical consistency, and format it appropriately for storage and further processing.
3. **Data Integration:** Ensure that extracted and structured data is readily accessible for further processing by other modules, such as the NLP-based query analysis.

2.2.5 NLP-based Query Analysis

1. **Query Processing:** Utilize NLP techniques to analyze user queries, extracting relevant keywords and integrating them with user profile data to refine search results.
2. **Result Ordering:** Order search results based on relevancy to user-specified criteria such as program type, location, and university ranking.
3. **Query Customization:** Offer users the option to select between rapid results and more detailed analyses based on their needs.

2.2.6 LLM Precision Tuning of Processed Data

1. **Model Training and Tuning:** Collect and maintain data on university programs and user profiles to train and fine-tune a large language model for accurate matching.
2. **Fit Score Calculation:** Develop a scoring system that normalizes and weighs user academic data and university requirements to calculate a fit score.
3. **Real-time Recommendation Generation:** Dynamically generate and highlight university recommendations based on user data, providing real-time updates to fit scores as user inputs change.

2.2.7 Priority URL Retrieval Engine

1. **Deep Analysis Response:** Respond to user requests for in-depth analysis by refining queries and conducting targeted searches for relevant data.

2. **URL Ranking:** Implement a TF-IDF based system to rank URLs by relevance to the refined user query, filtering out less relevant links.
3. **Efficient Data Retrieval:** Use web crawling tools to efficiently locate and retrieve data from identified URLs for further processing.

2.2.8 Application Deadline Tracker

1. **Deadline Monitoring :** Periodically monitor university websites to track application deadlines and significant updates using web crawling techniques.
2. **Automatic Notifications:** Send automated notifications to users about impending application deadlines relevant to their profiles and interests.

2.2.9 Database Management and Data Storage

1. **Database Schema Design:** Design and implement a flexible NoSQL database schema to manage diverse data types, including user profiles, university information, and scraped data.
2. **Efficient Data Handling:** Provide efficient data ingestion, retrieval, and management functionalities to handle CRUD operations swiftly and reliably.

2.3 Non-Functional Requirements

2.3.1 Reliability

- A failure is defined as any circumstance that stops the user from carrying out an intended action, such updating a profile or retrieving data, and the system should have a Mean Time Between Failures (MTBF) of at least 300 operational days.
- In event of a system breakdown, an automatic recovery procedure should restore functionality within 5 minutes 95% of the time.
- To find and fix problems early on, the system should include regular reviews and ongoing error tracking.

2.3.2 Usability

USE-1: On all devices with a steady internet connection, the user interface should load in two to four seconds, guaranteeing speedy access and engagement.

USE-2: Reusable UI components in React improve development efficiency by preserving consistency and cutting down on development time.

USE-3: To guarantee that all user interface elements are accessible to those with impairments, the system has to be compatible with the ARIA standards.

2.3.3 Performance

PER-1: For 95% of all actions, the system should finish data processing and user profile populating from submitted CVs in 10 to 15 seconds.

PER-2: When operating normally, the web scraper has to finish scraping any university website in 30–60 seconds.

PER-3: Within seconds of data extraction, the smart data identifier should validate that retrieved entities from the CV PDF files have an accuracy rate of 85% or above.

2.3.4 Security

In order to prevent unwanted access to user data, the system must strive for an annual security breach probability of less than 0.01%. With a security effectiveness of 99.9%, data integrity must be preserved by implementing procedures that guarantee that only authorized users can alter the data.

2.4 Domain Model

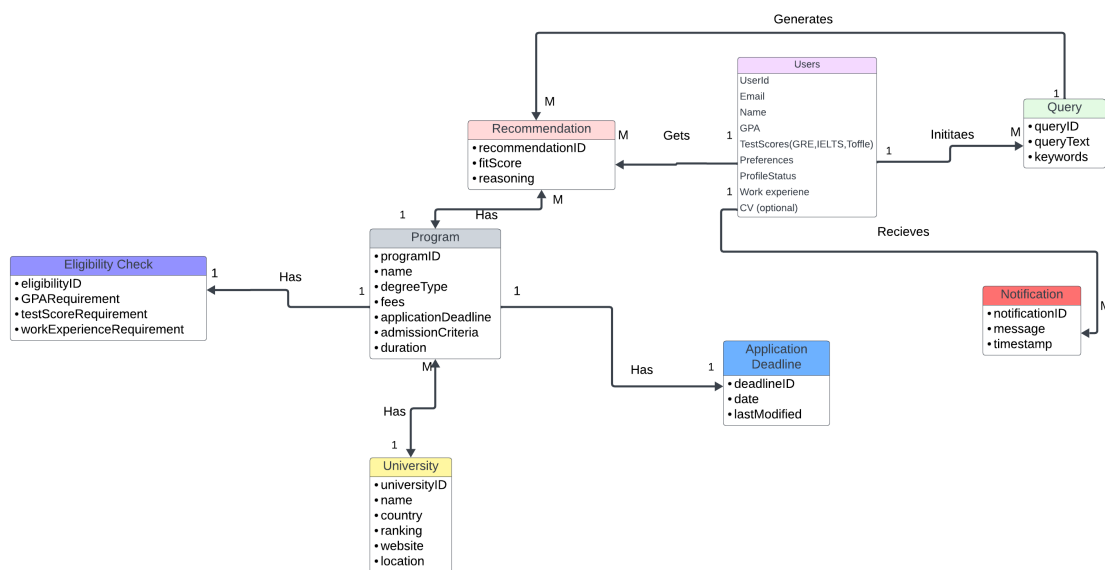


Figure 2.2: Domain Model Diagram of EduVantage

Chapter 3

System Overview

The EduVantage system's architecture is built to effectively manage the complexity of connecting students with appropriate graduate programs according to their choices and characteristics. The Presentation Layer, Business Logic Layer, and Data Layer are the three tiers of the presentation architecture used by the system. This division makes sure that data management, business processing, and user interface management are all handled separately yet work together harmoniously.

3.1 Architectural Design

3.1.0.1 Architectural Layout

- **Presentation Layer:** Users interact with the system at this front-end layer. A web-based client that offers an easy-to-use interface for accessing all system features is used to build it.
- **Business Logic Layer:** This layer functions as the system's central processing hub, where all choices, logic, and business rules are carried out. Serving as a bridge between the Presentation Layer and the Data Layer, it executes application logic and guarantees data integrity.
- **Data Layer:** the back-end layer, which manages, stores, and retrieves data. It stores data in a schema-less fashion using MongoDB, a NoSQL database, enabling flexibility in managing the range of data types utilized by the system.

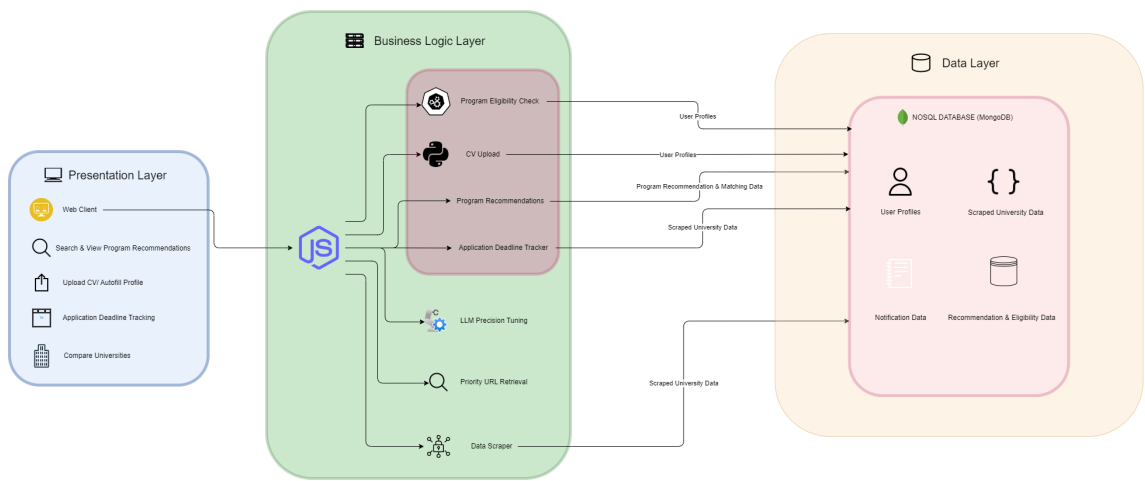


Figure 3.1: Architecture Design

3.2 Design Models

3.2.1 Activity Diagrams

Search and view Recommendations

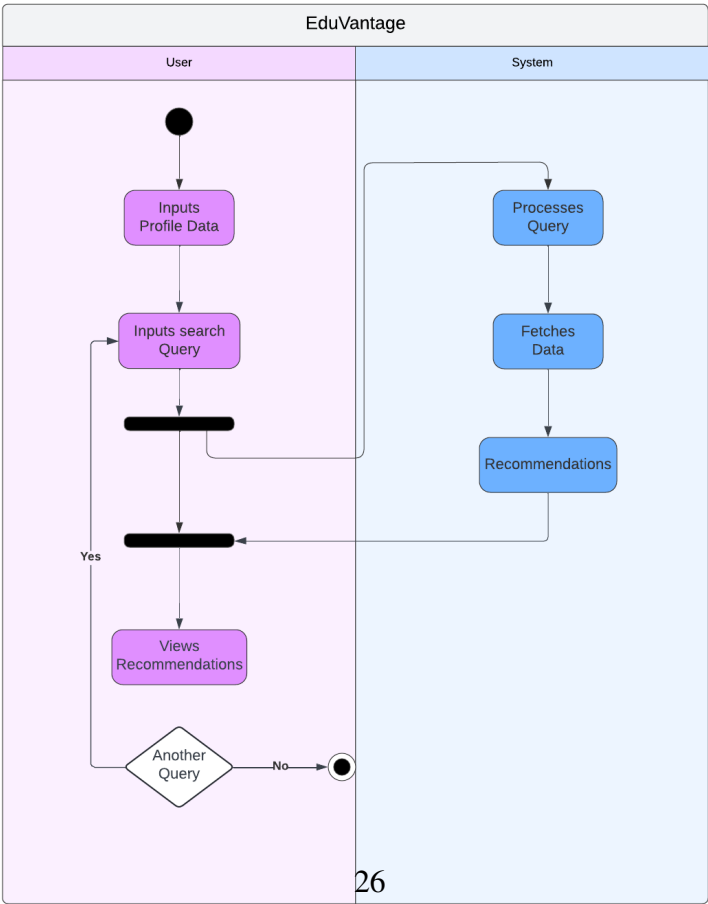


Figure 3.2: Activity Diagram for Search and view Recommendations

Upload CV and auto filling of profile

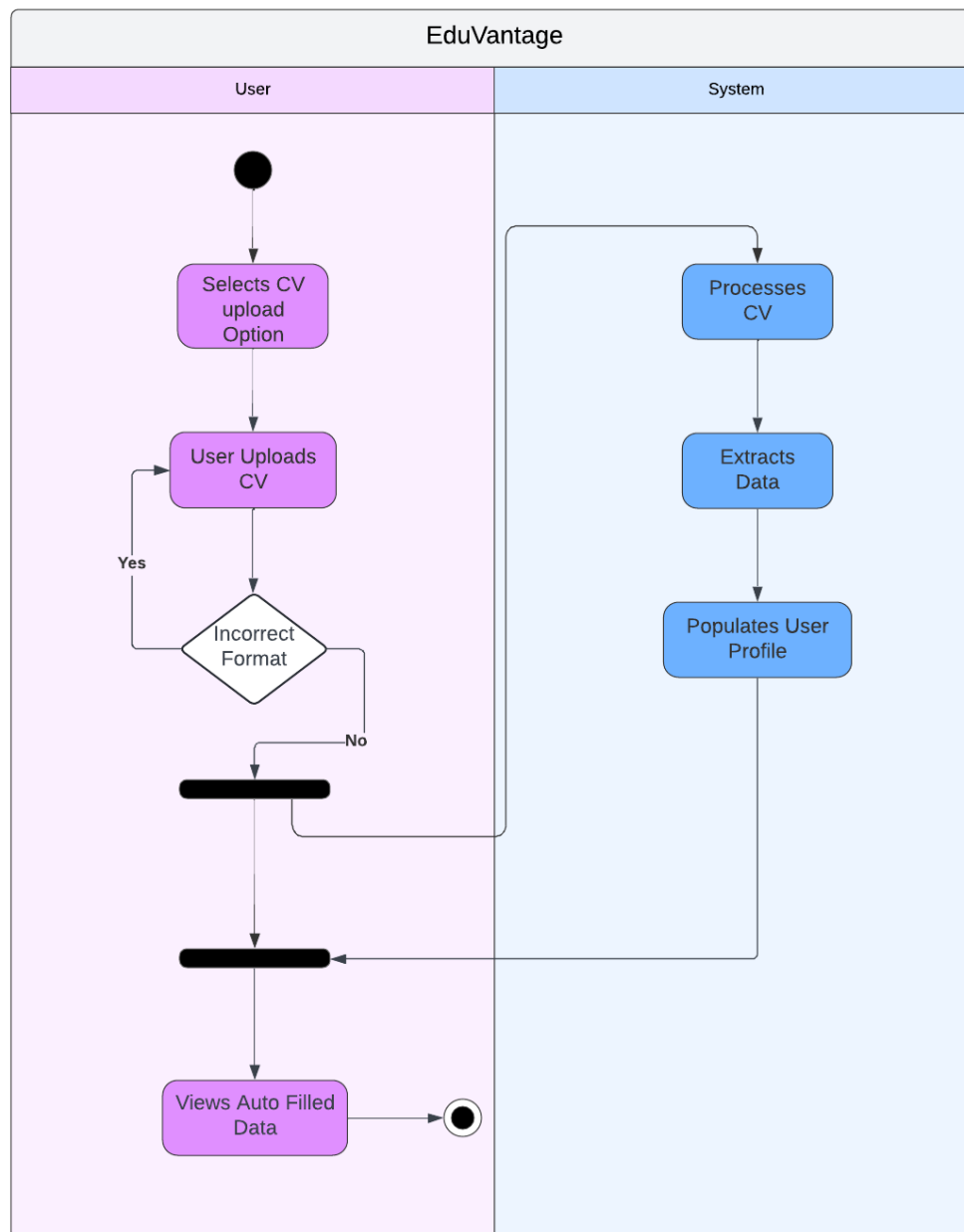


Figure 3.3: Activity Diagram for Upload CV and fill profile

Check program eligibility

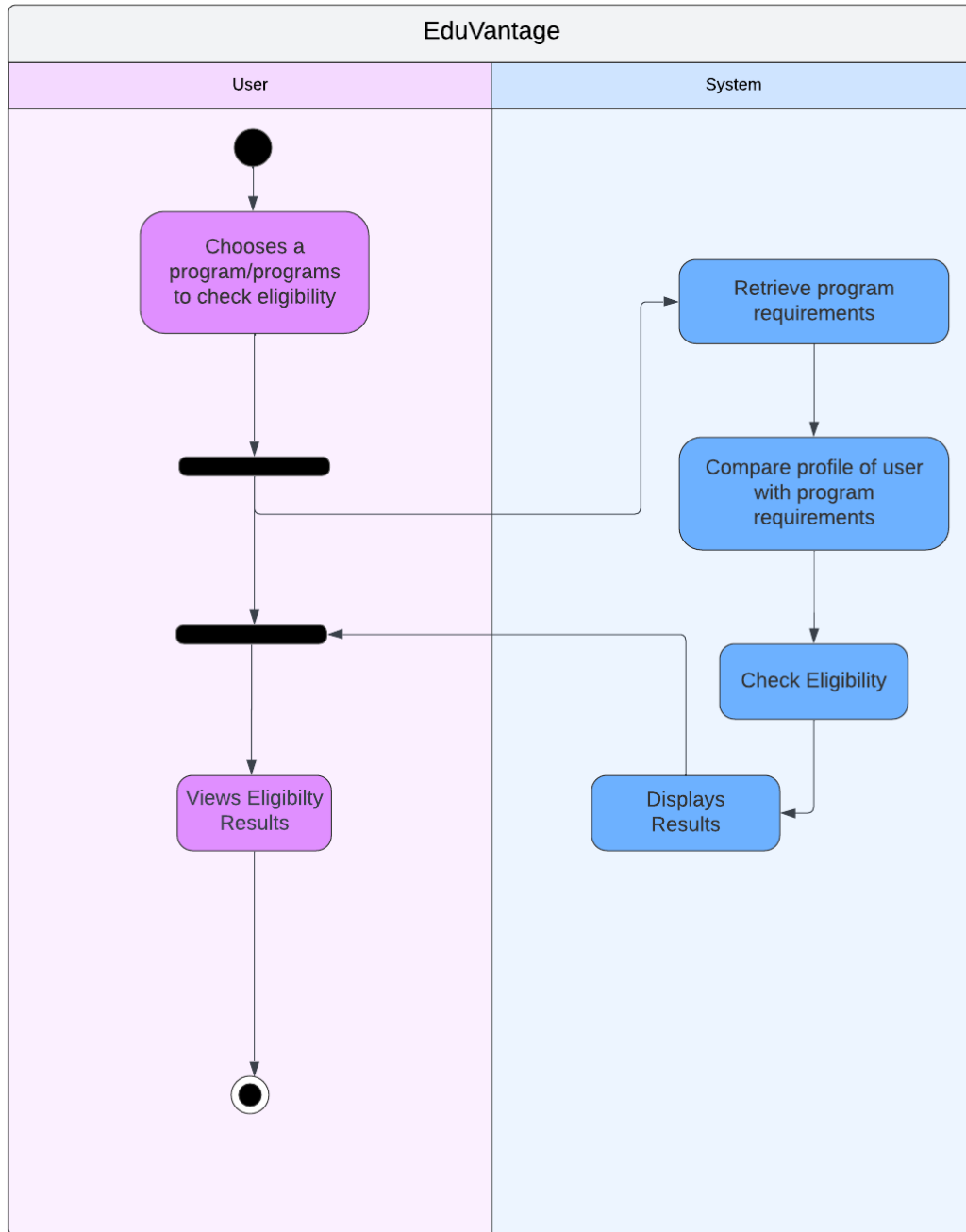
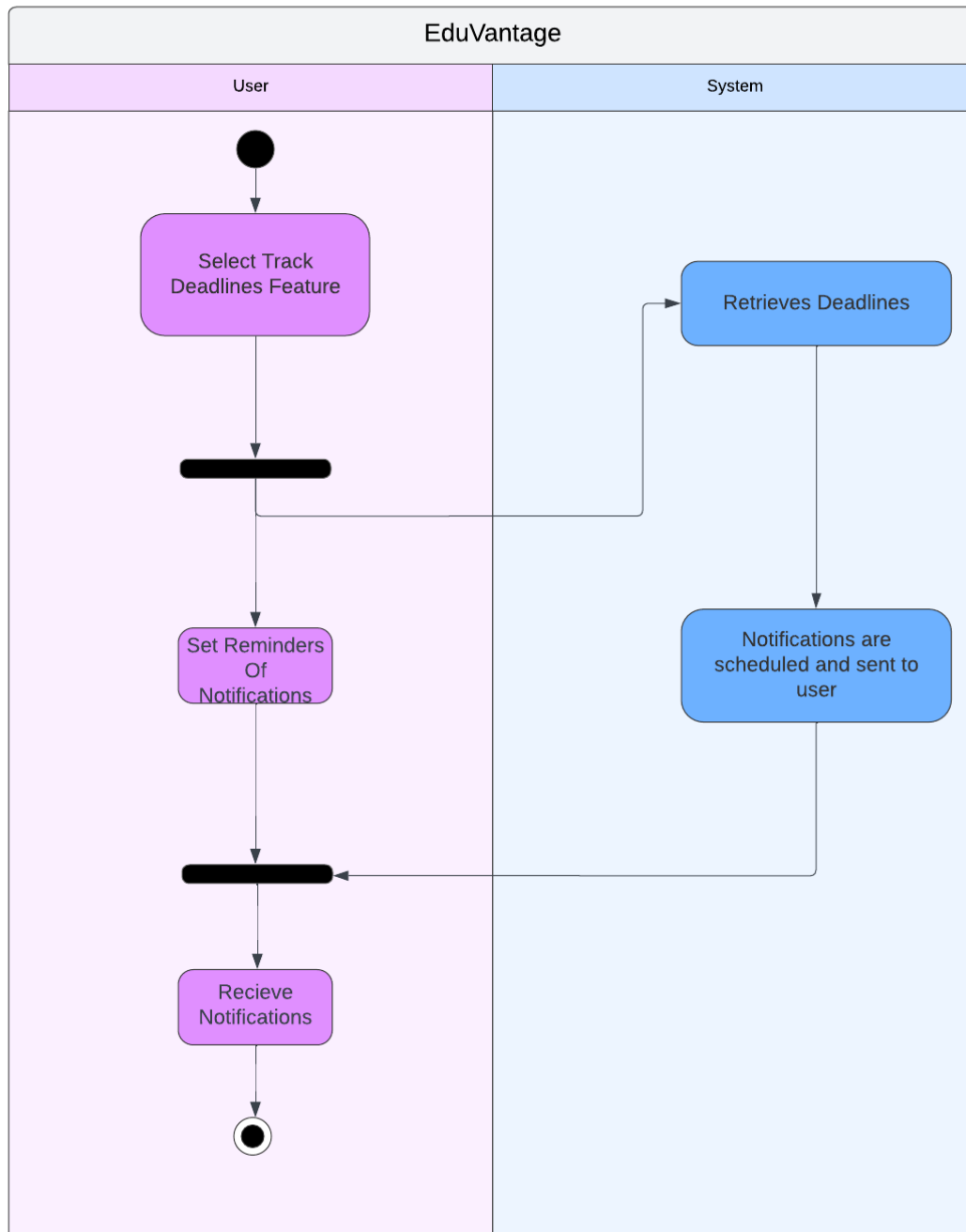


Figure 3.4: Activity Diagram for Check Program Availability

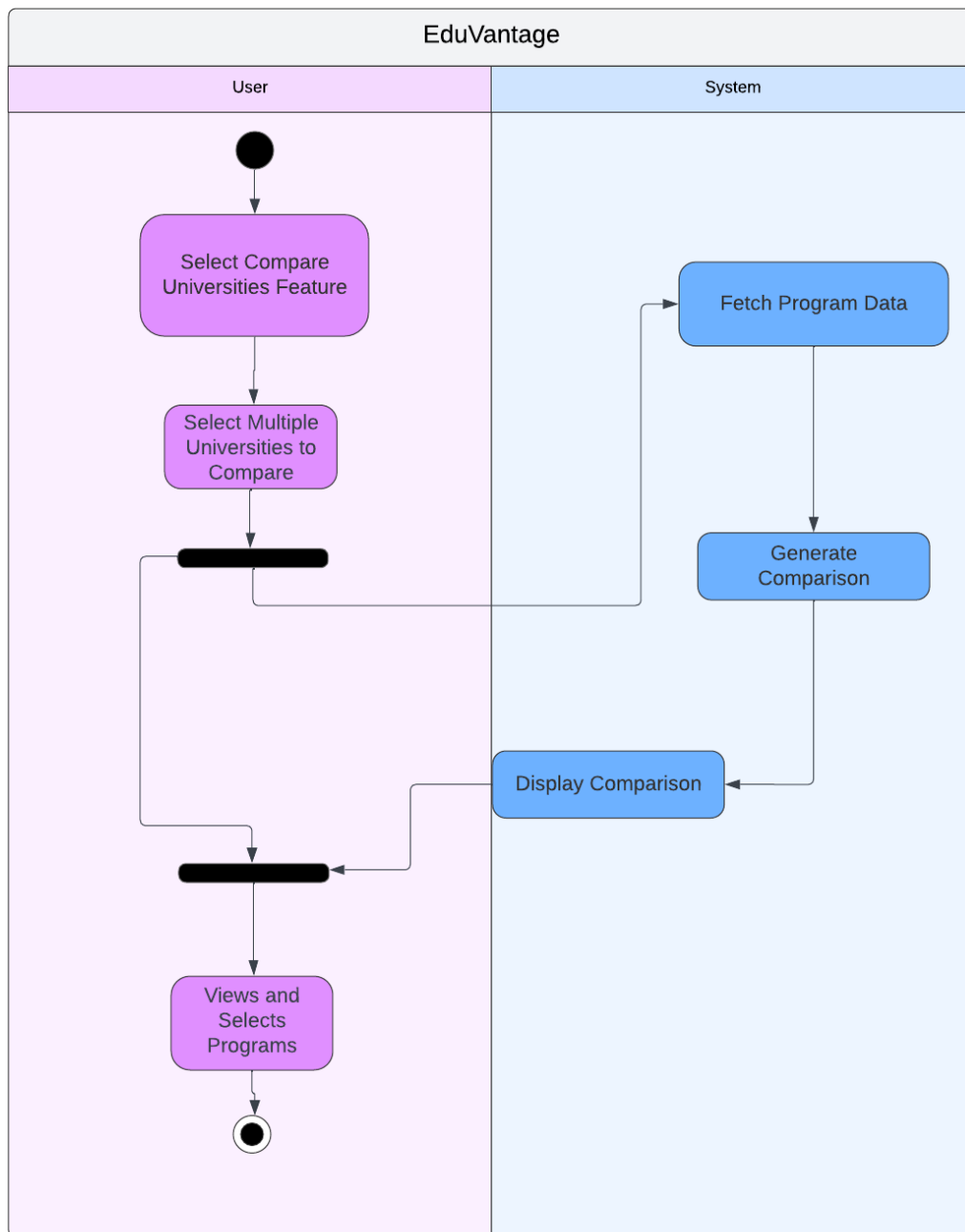
Track Application Deadline



Activity Diagram for

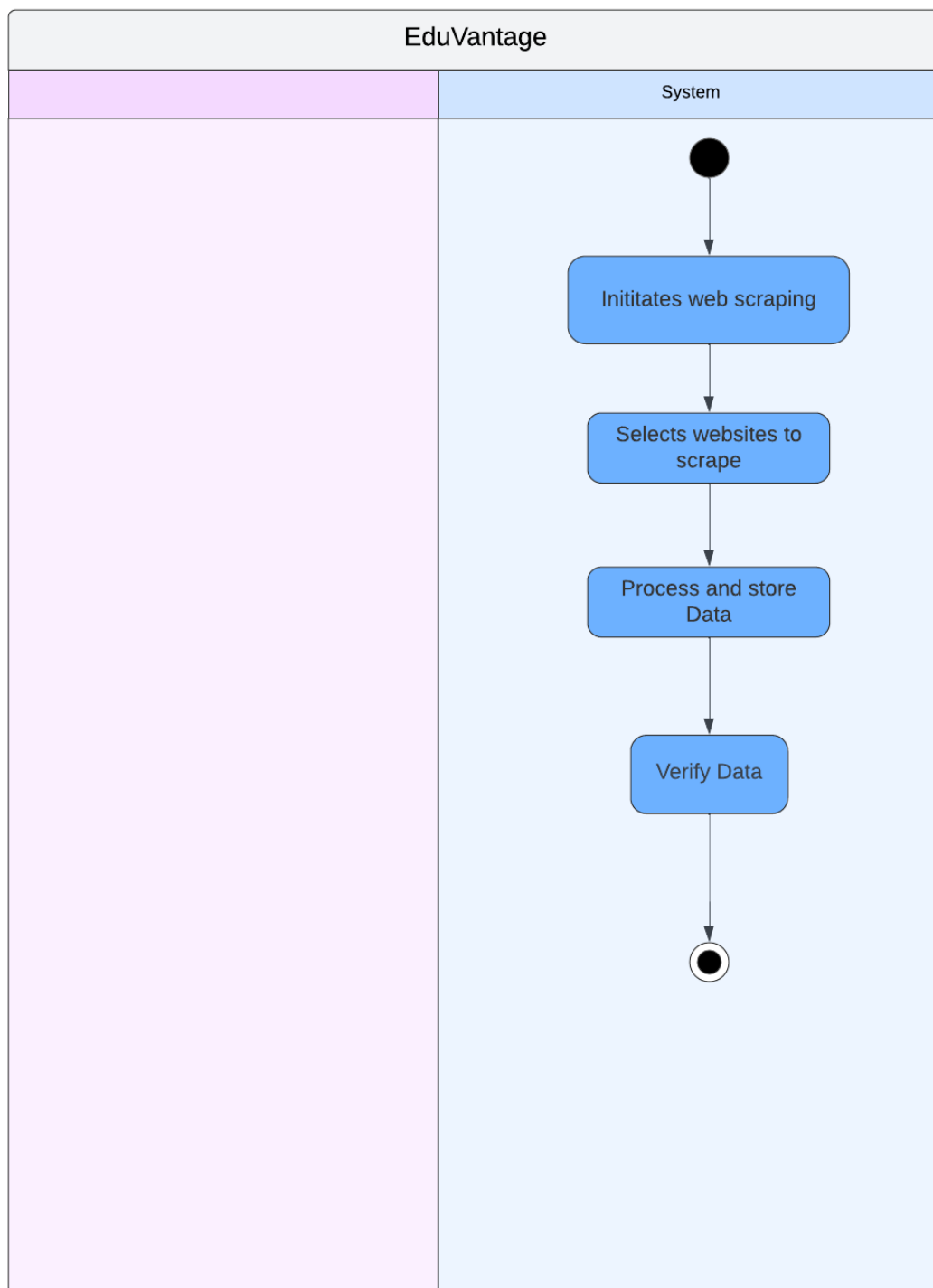
Figure 3.5: Track application deadline

Compare Universities



Activity Diagram for
Figure 3.6: Compare Universities

Storing and Scraping Data from System side



Activity Diagram for

Figure 3.7: Storing and Scraping Data

3.2.2 System-level Sequence Diagram

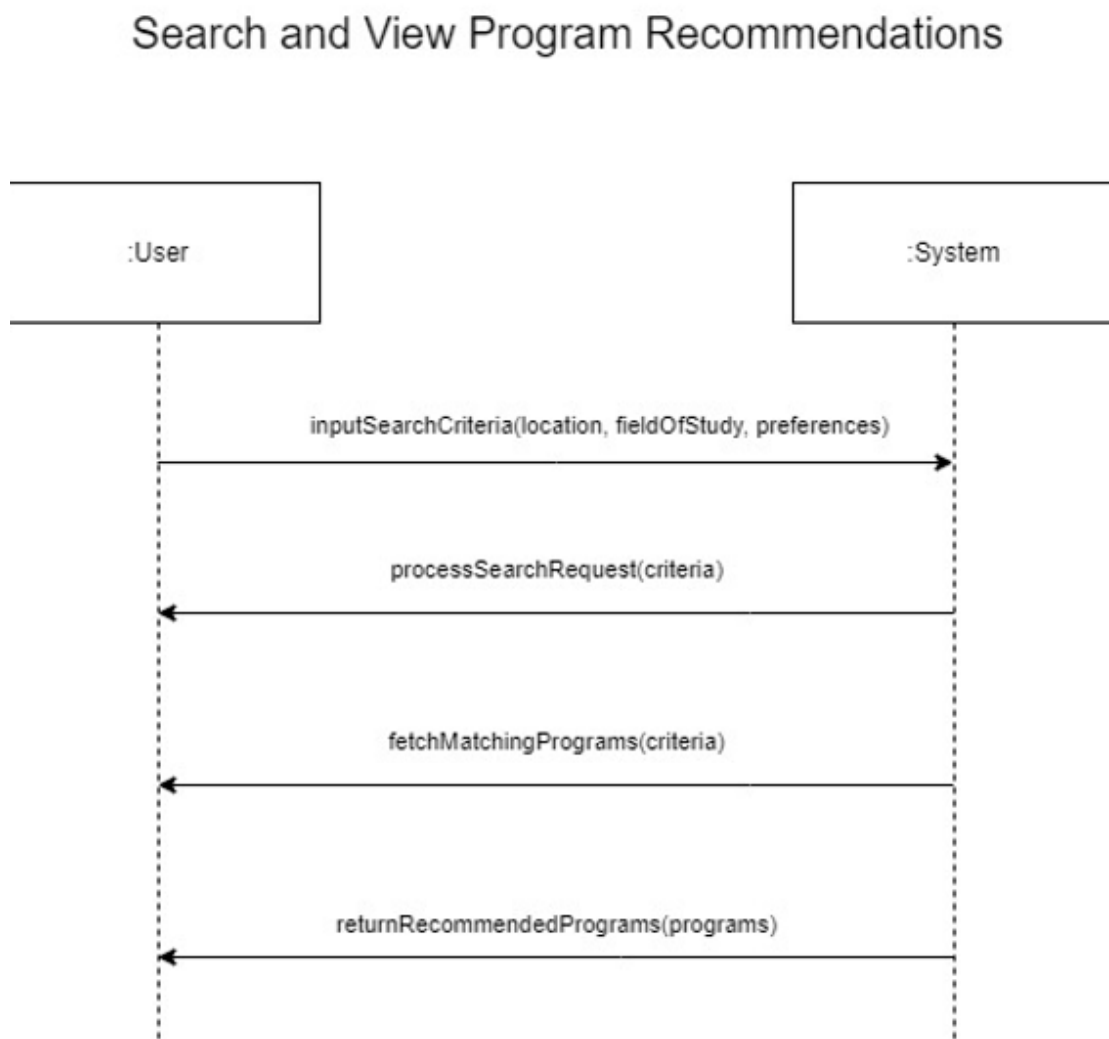


Figure 3.8: SSD of "Search and View Program Recommendations"

Upload CV and Autofill Profile

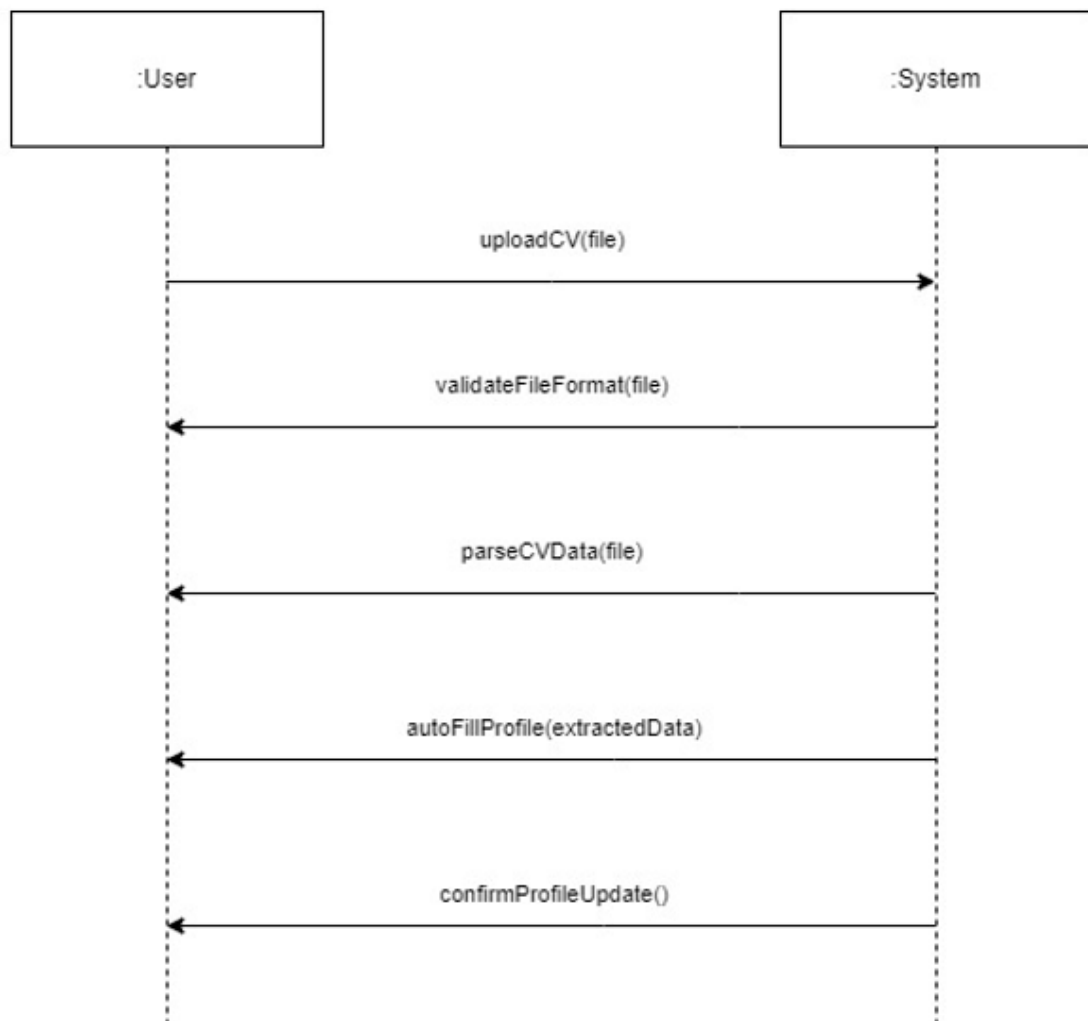


Figure 3.9: SSD of "Upload CV and Autofill Profile"

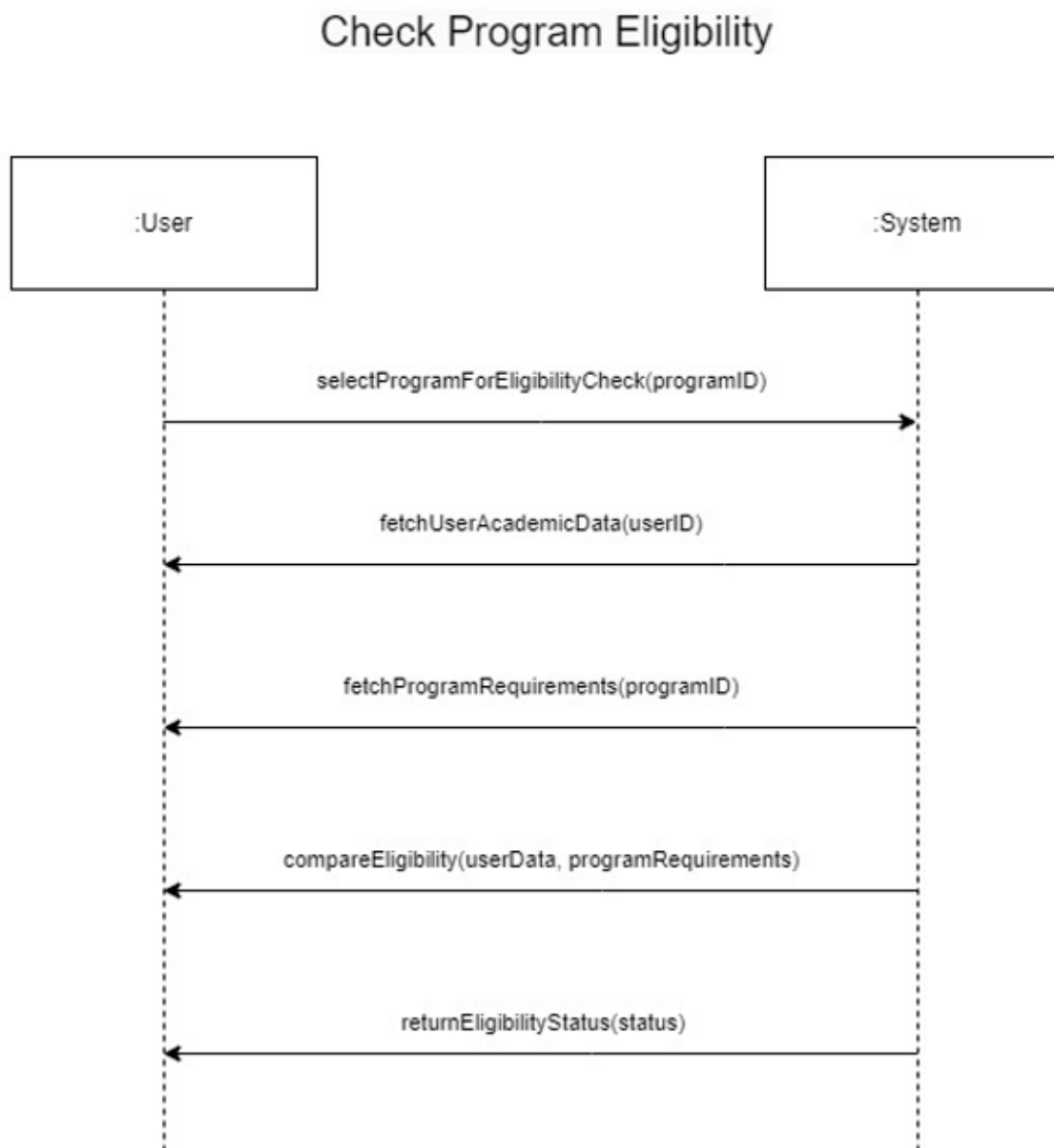
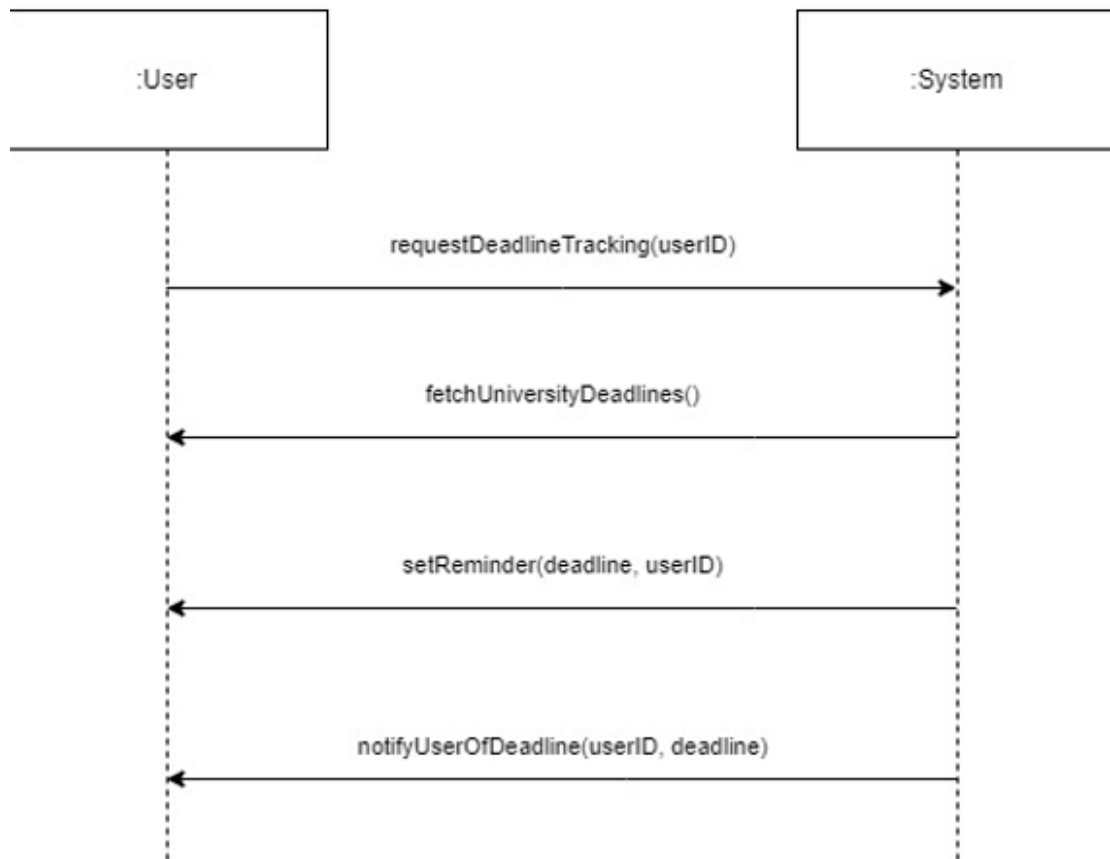


Figure 3.10: SSD of "Check Program Eligibility"



Track Application Deadlines

Figure 3.11: SSD of "Track Application Deadlines"

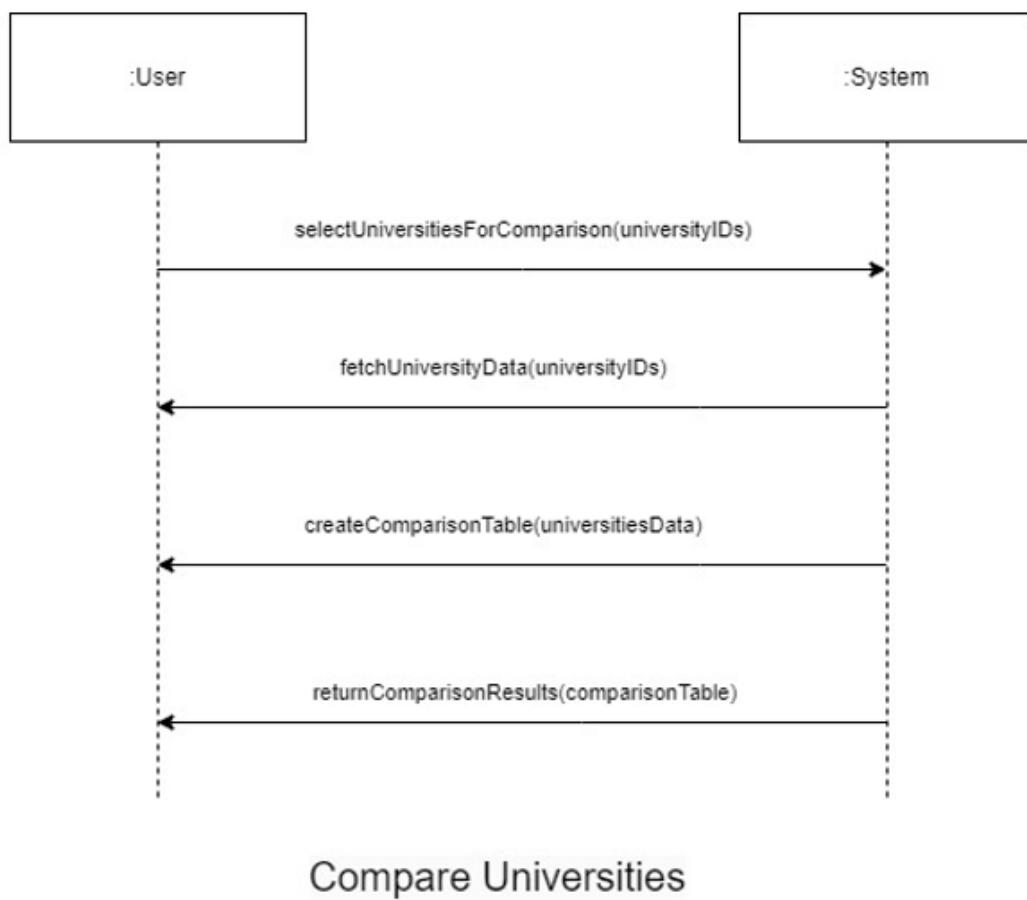


Figure 3.12: SSD of "Compare Universities"



Figure 3.13: SSD of "Scrape and Store University Data"

Search and View Program Recommendations: The "Search and View Program Recommendations" function of the EduVantage system's System Sequence Diagram (SSD) illustrates the process of user-system interaction. The system records and processes the user's initial input of search parameters, such as location, area of study, and personal preferences. In order to get matching programs from the database or an internal API, the data must first be validated and prepared. The user receives a list of suggested programs from the system when it has successfully retrieved the pertinent program data based on the given criteria. This SSD demonstrates the dynamic interaction required to provide customized educational content by clearly outlining the data flow from user input to the delivery of customized program suggestions.

Upload CV and Autofill Profile: The user uploads their resume and it is evaluated for format (PDF or DOCX) via a simplified procedure described in the System Sequence Diagram (SSD) for the "Upload CV" function of the EduVantage system. Following

validation, the system extracts crucial data from the CV, including the user's personal information and educational history, and automatically completes their profile. Ultimately, the system verifies that the user's profile has been updated successfully. This effectively completes the CV upload and integration process, minimizing the need for human data entry and improving the accuracy of user profiles.

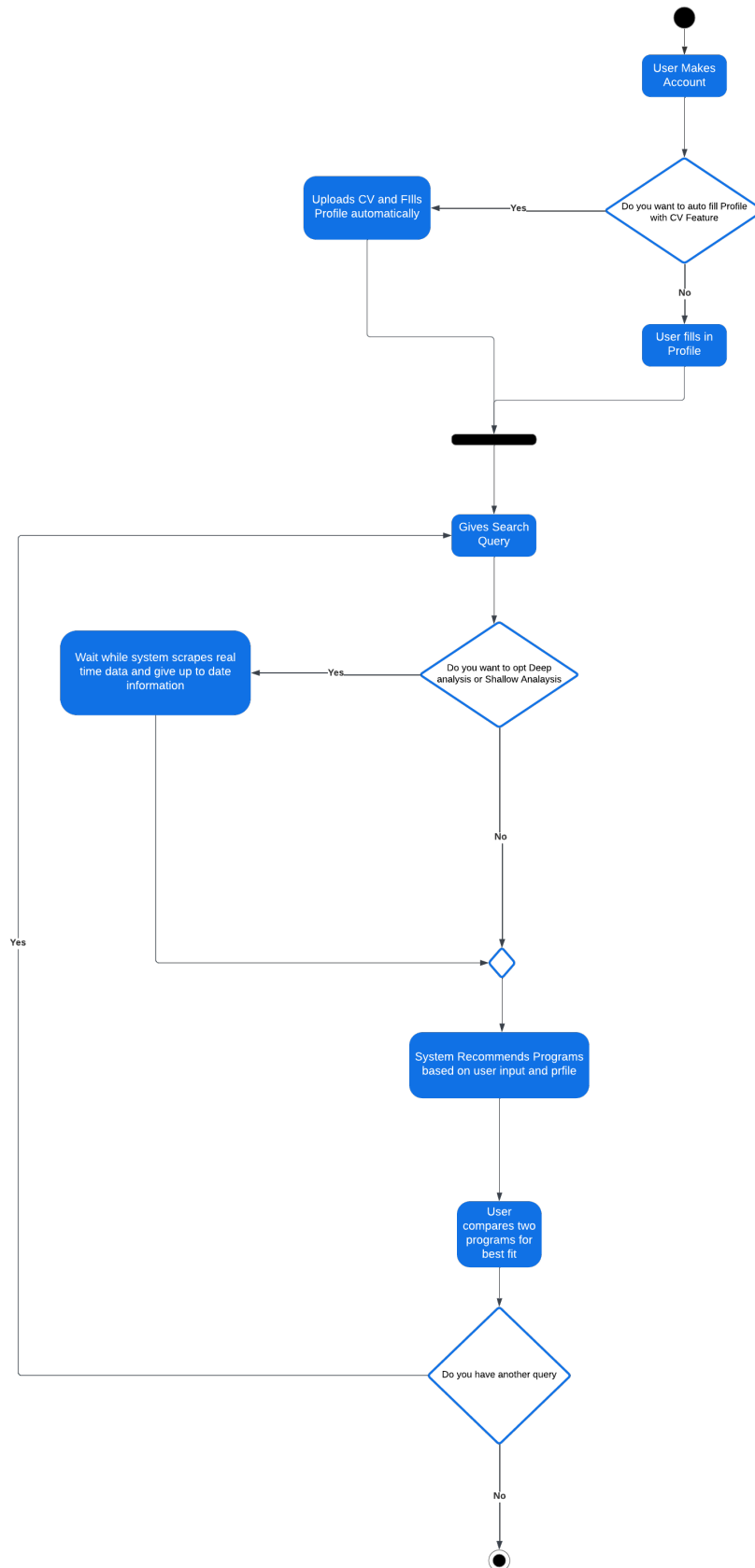
Check Program Eligibility: The procedure by which a user chooses a program to ascertain their eligibility is described in the System Sequence Diagram (SSD) for the EduVantage system's "Check Program Eligibility" function. The user's academic records and the prerequisites for the program are retrieved by the system. After that, it contrasts the two to determine eligibility. By automating eligibility checks for academic programs, the system improves the user experience by promptly and accurately informing the user of their eligibility status.

Track Application Deadlines: The System Sequence Diagram (SSD) for the "Track Application Deadlines" feature in the EduVantage system illustrates how the system helps users manage university application deadlines. Initially, a user triggers the process by requesting deadline tracking. The system then fetches the relevant deadlines and sets reminders for the user. Finally, it notifies the user of each impending deadline through automated alerts, ensuring they are well-informed and can prepare their applications timely. This automated process significantly reduces the manual effort required by users to track important deadlines, thereby enhancing the overall user experience.

Compare Universities: The System Sequence Diagram (SSD) for the "Compare Universities" feature in the EduVantage system maps out the steps involved when a user wishes to compare various universities. The process begins when a user selects multiple universities for comparison by providing their unique identifiers. The system then retrieves data for these selected universities. Once the data is fetched, the system constructs a comparison table, organizing the information in a way that facilitates easy comparison across different criteria such as tuition, programs offered, faculty, etc. Finally, the comparison results are presented to the user in the form of this structured comparison table. This feature assists users in making informed decisions by directly comparing different educational institutions on various important factors.

Scrape and Store University data: The System Sequence Diagram (SSD) for "Scrape and Store University Data" depicts an administrative process within the EduVantage system. An admin initiates this process, which involves scraping university data from specified URLs using an external system. After scraping, the data is stored in a database, and the system confirms the successful storage of this data back to the admin. This SSD highlights the automated updating of university information, ensuring the system provides users with current and accurate data.

3.2.3 State Transition Diagram



3.2.4 Data Flow Diagram

- User Interaction

User: This would be any end user of the system, students, or even academic advisors using the system. **Profile Management:** Users manage their profiles, which involves inputting and updating personal and academic information. This can be done manually via forms or automatically through CV uploads.

- Data Input Data Processing

CV Upload: Users will be able to upload their CVs, after which the system will automatically fill out their profiles by extracting relevant data from the CV. **Fill through forms:** Alternatively, users can fill in their information using web forms.

- Data Processing and Analysis

Prompt: Users generate prompts for the system, such as searching for university programs or asking for program recommendations. **Standard Data Scan:** For quick results, the system uses pre-existing data that has been previously scraped and stored in the database.

Comprehensive Data Scan: This system scans overall data in its comprehensive scan, rescraping the same data for the latest information for the detailed inquiries. **Tokenization and matching of TF-IDF:** During processing, the system tokenizes the text data and performs Term Frequency-Inverse Document Frequency (TF-IDF) techniques to deduce and retrieve the most relevant information according to the query entered by the user.

Web Scraping: Automatically collects fresh data from university websites to ensure information is always up-to-date. **NER (Named Entity Recognition):** Using NER techniques, the unstructured data that is scraped from the web gets structured.

Scheduled Web Scraper: Regularly scheduled scraping tasks refresh the database to ensure that the stored data is current.

Update DB: The processed and structured data in the web scraping and NER operations are further stored in the MongoDB database.

- Recommendation and Notification

RAG (Retrieval-Augmented Generation): Enhances the recommendation process by incorporating contextual data retrieval into the response generation, leveraging a large language model (LLM) that is fine-tuned to provide precise and personalized suggestions.

Fine-Tuned LLM: The LLM is fine-tuned with specific data to generate highly personalized academic program recommendations and related information such as scholarship opportunities. Personalised Programs and Scholarship Recommender Based on the analysis conducted by the LLM, personalized academic and scholarship recommendations are made available to the user.

Deadline Reminder: Ensures users are aware of upcoming application deadlines, facilitating timely applications to programs of interest.

- Data Storage

Database (MongoDB): Serves as the central depository of all data, including user profiles, university program data, and system logs. It stores structured data to support speedy information retrieval for different functionalities within the system.

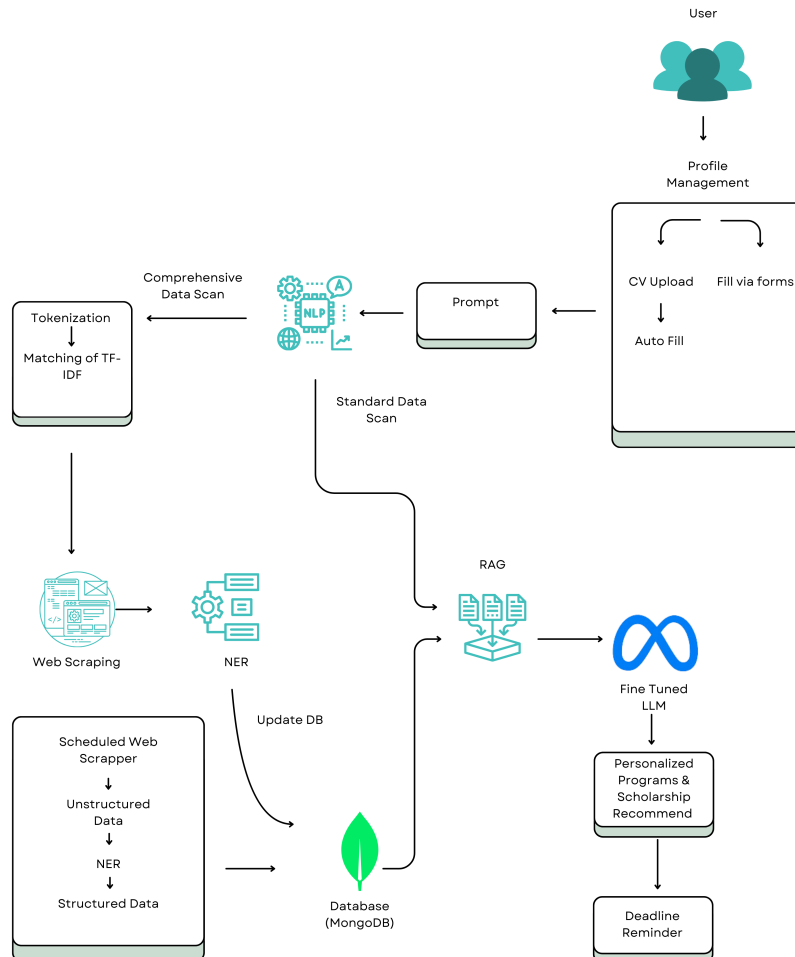


Figure 3.15: Data Flow Diagram

3.3 Data Design

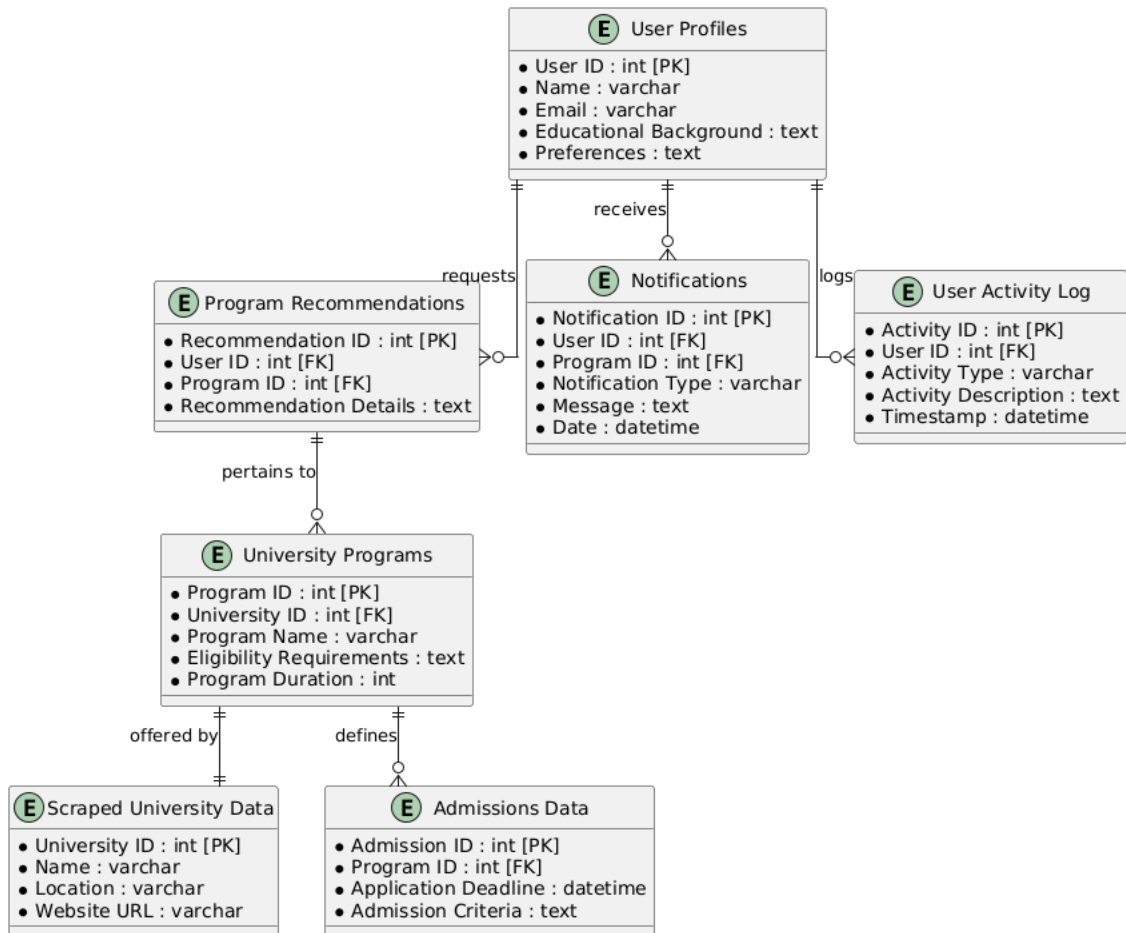


Figure 3.16: Entity Relationship Diagram

Transformation of information into data structures

1. User Profiles

- **Structure:** Every user profile has areas that record academic and personal data, including name, email, educational background, and interests.
- **Processing:** The system uses parsing algorithms to retrieve pertinent information from a user's uploaded CV and populate the user profile fields. Additionally, users can manually update or enter their data.
- **Storage:** User profiles are stored in a NoSQL database, MongoDB, allowing flexible schema to accommodate varied data without requiring a fixed structure.

2. Program Recommendations

- **Structure:** This includes recommendation IDs linked to user IDs, a list of recommended programs, and detailed program information derived from matching algorithms.
- **Processing:** The system uses a combination of user profile data and university program data to generate personalized recommendations through algorithms that consider user preferences and program criteria.
- **Storage:** Recommendations are stored in MongoDB with references to both user profiles and program data, ensuring quick access and update capabilities.

3. University Programs

- **Structure:** Program entities include program IDs, related university IDs, program names, eligibility requirements, and other relevant details.
- **Processing:** Data about university programs is scraped from university websites, processed for relevance and accuracy, and then structured into program entities.
- **Storage:** Program data is stored in MongoDB, linked to the respective university data entities to maintain data integrity and coherence.

4. Scraped University Data

- **Structure:** Includes university IDs, names, locations, and URLs where more detailed information can be retrieved.
- **Processing:** Automated scraping tools regularly extract and update information from university websites. This data is then cleaned and structured into the university data entities.
- **Storage:** Stored in MongoDB, ensuring that data can be efficiently queried and linked to other relevant entities such as university programs.

5. Notifications:

- **Structure:** Comprises notification IDs, associated user IDs, messages, and other notification details like dates and types.
- **Processing:** The system generates notifications based on user actions or system triggers (like approaching application deadlines).
- **Storage:** Notifications are stored in MongoDB and are directly linked to user profiles for timely delivery and personalization.

6. Admission Data

- **Structure:** Contains admission IDs, program IDs, application deadlines, and specific admission criteria.
- **Processing:** Admissions data is updated periodically through data scraping or manual updates to reflect the most current information.
- **Storage:** Stored in MongoDB, linked to specific university programs to facilitate easy access and reference during the application process.

Data Storage Items:

- **MongoDB:** A NoSQL database used for storing all the entities mentioned. It supports a flexible schema approach which is ideal for handling the variety of data formats and structures encountered in educational data.

Bibliography

- [1] Yujuan Ding, Wenqi Fan, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. A survey on rag meets llms: Towards retrieval-augmented large language models. *arXiv*, 2405.06211v1, 2024.
- [2] Donny. Upload pdf files to mongodb with expressjs, multer, nodejs. <https://stackoverflow.com/questions/69775749/upload-pdf-files-to-mongodb-with-expressjs-multer-nodejs>, 2021. Accessed: 2024-09-12.
- [3] Google. Material-ui. <https://mui.com/material-ui/>, 2014.
- [4] Yixin Liu, Avi Singh, C. Daniel Freeman, John D. Co-Reyes, and Peter J. Liu. Improving large language model fine-tuning for solving math problems. *arXiv preprint arXiv:2310.10047*, 2023.
- [5] Jacob Thornton Mark Otto. Bootstrap. <https://getbootstrap.com/>, 2010.
- [6] Fabian Pedregosa and Varoquaux. Scikit-learn: Machine learning in python. <https://scikit-learn.org/stable/>, 2011. Accessed: 2024-09-12.
- [7] Charles Rea. jsonwebtoken. <https://www.npmjs.com/package/jsonwebtoken>, 2010.
- [8] Andris Reinman. Nodemailer. <https://nodemailer.com/>, 2010.
- [9] Rik Schennink. Filepond documentation: Installation - react. <https://pqina.nl/filepond/docs/getting-started/installation/react/>, 2024. Accessed: 2024-09-12.
- [10] Sideway. joi. <https://www.npmjs.com/package/joi>, 2024. Accessed: 2024-09-12.
- [11] Chrome Dev Tools Team. Puppeteer. <https://pptr.dev/>, 2017.
- [12] TopUniversities. Qs world university rankings, 2024. Accessed: 2024-09-13.
- [13] UBI AI. Exploring nlp techniques: Tokenization, pos tagging, and ner. <https://medium.com/ubiai-nlp/exploring-nlp-techniques-tokenization-pos-tagging-ner>, 2022. Accessed: 2024-09-12.